Confidential and verifiable upstream inference over standard TLS 1.3

A no-trusted-hardware protocol for LLM routing nodes

routstr-confidential project

2026-10-10

Abstract

LLM routing nodes forward user requests to upstream API providers with the node's own credentials. Users must today trust the node with their prompts, and providers' billing is opaque to them. We present a protocol in which the user (client) is the TLS 1.3 client of the upstream provider and the node is only a TCP relay. For exactly one key epoch the node receives one-shot record material — proven in interactive zero-knowledge to derive from the session's handshake secret — that lets it write exactly one TLS record: the request head carrying its `Authorization` header. The prompt body and the streamed response then run under application keys only the client holds, at native speed. The node verifies the upstream's certificate and handshake; the client receives machine-checkable proofs that the request carries the agreed public template and cost-pinning suffix, and that the usage and cost figures come from the upstream itself. The design uses standard TLS 1.3 and VOLE-based ZK, requires no trusted hardware and no MPC over the payload, and was verified end-to-end against a live provider with all proofs checking: measured added latency over a direct HTTPS call is about +1.1 s to first token for a cold session and about +0.3 s for a pre-warmed one (client and node on one machine; Section 6 gives WAN figures). Each key-schedule proof takes about 0.45 s and 1.6 MB locally, the head-template proof about 0.09 s and 0.95 MB. We give the provider profile that determines applicability, analyze the security properties and residual risks, and compare with TLSNotary-style MPC-TLS, proxy zkTLS attestors, and enclave (TEE) approaches at the end.

Contents

1. Introduction **2**
1.1 Contributions . . . 2

2. Model **3**

3. Preliminaries and the provider profile **3**
3.1 Provider profile . . . 3
3.2 TLS 1.3 mechanics used . . . 4

4. Protocol **4**
4.1 Epoch structure . . . 4
4.2 Handshake, relayed and doubly verified . . . 5
4.3 Credential record (epoch 1) and the key-material proof . . . 5
4.4 Body (epoch 2) . . . 6
4.5 Response and usage disclosure . . . 6
4.6 Settlement . . . 7
4.7 Abort handling . . . 7

5. Security analysis **8**

6. Implementation and evaluation **10**
6.1 Prototype . . . 10
6.2 Costs and optimizations . . . 10
6.3 Live verification . . . 11

7. Deployment pattern (negotiation, billing, compatibility) **11**

8. Comparison with alternative approaches (at the end) **12**

9. Conclusion **13**

Appendix A. Case study: the provider profile in practice **14**

1. Introduction

Routing nodes and gateway services sit between users and upstream LLM APIs. A node holds an upstream account and forwards requests with its own API key; users pay the node (typically prepaid with e-cash tokens such as Cashu), and the node pays the upstream. Two trust problems follow.

The user trusts the node with everything. The node sees the prompt and the response; it can log, alter or replay them. This is a privacy problem (prompts contain source code, contracts, medical text), and a correctness problem (the node can substitute models or parameters).

The node trusts the user with nothing, and vice versa on billing. The node's upstream API key must never leak to users. Charging must be defensible: the user should be able to check that the tokens billed are the tokens the upstream actually produced and reported.

Existing zkTLS-based "verified" modes put the node in the TLS client seat and have a notary/verifier check its transcripts. That proves what left the node but cannot hide the prompt from the node, and the MPC variants are slow (tens of seconds, hundreds of megabytes). Enclave designs (running the node in a TEE) remove the operator from the trust equation but introduce hardware-vendor trust and an attestation lifecycle that users must chase after every update.

This paper describes a different point in the design space: **hand the TLS client role to the user and let the node relay ciphertext**, with cryptographic proofs that pin the few bytes the node is allowed to write and bind billing to the upstream's own numbers. It requires neither trusted hardware nor MPC over the payload, streams the response as it arrives, and adds about a second to first token in our prototype (about 0.3 s with pre-warmed sessions).

1.1 Contributions

1. A TLS 1.3 protocol (section 4) in which a relay node writes exactly one request record — its credential — with per-record material proven in ZK against the session's handshake secret, after which the client rotates send keys twice so the node's material is one-shot and useless for content.
2. A request-integrity construction: the client proves that its request body ends in an agreed public JSON suffix that pins every cost-relevant parameter (section 4.4), and receives a ZK proof that the hidden head matches a published template (section 4.3).
3. A verifiable-accounting construction: the client proves that the upstream's terminal usage/cost event is authentic ciphertext output of the session, anchored at the end of the relayed transcript (section 4.5), and the node charges exactly that usage, at the rates of the offer the client verified, through its ordinary billing (section 4.6). The suffix and usage proofs reveal one bit (equal or not) to the node, never decrypted bytes.
4. A **provider profile** (section 3.1) that separates what any upstream must satisfy from provider-specific details, making the design portable beyond the case-study provider.
5. A working implementation and evaluation (section 6): all four proofs verified against a live provider; about +1.1 s (cold) and +0.3 s (pre-warmed) time-to-first-token versus a direct HTTPS call on one

host; ~0.45 s and ~1.6 MB for each of π_{C1}, π_{C2}, π_{C3}, ~0.09 s and ~0.95 MB for π_N, ~5.9 MB per session in both directions.

6. A deployment pattern and capability negotiation for nodes and their client daemons (section 7) that is additive and stateless, and a comparison with alternative approaches (section 8).

2. Model

Parties:

- **C — client** (a user's daemon or app). TLS 1.3 client of the upstream session; the only holder of the application traffic secrets; payer of the node.
- **N — node**. Holds the upstream API key; owns the TCP connection to the upstream and relays it; is the only party that writes the credential record; bills C through its ordinary accounts.
- **U — upstream** (any provider whose profile, section 3.1, is satisfied). Unmodified.

Channels: C to N is a control channel plus a byte relay (in our deployment, WebSocket); N to U is ordinary TCP:443.

Goals:

ID	Property
P1	Credential confidentiality: only N knows the upstream API key, under the egress assumption stated in section 5.
P2	Content confidentiality: only C (and U) can read the prompt and the response. N sees ciphertext plus the public billing data: the usage event, or on an upstream error the response head.
P3	Integrity: C can verify the request U received is C's own content plus the agreed public template content (not its exact wire bytes, residual risk 1), and that the response is exactly what U sent (tool calls included).
P4	Verifiable accounting: the charge is computed from the token usage the upstream itself reported in the session, verified by N (who settles) and by C (who checks the signed receipt against its own plaintext).
P5	Upstream authenticity: both parties verify the session terminates at a server holding a valid certificate for the upstream hostname.
P6	Freshness: every session is live and bound to one payment; nothing can be reused across sessions.

Non-goals: hiding the prompt from U (U processes it by definition); proving which model weights ran; hiding traffic volume and timing from N (but see the discussion in section 5); and third-party verifiability. The proofs convince the two parties of the session, which is what settlement needs. They are interactive and designated-verifier, and C, holding the server's application keys, could forge server records for anyone but N. A receipt is N's signed statement, not a proof to outsiders.

Trust assumptions: C trusts its own device; N trusts its own device and the upstream's accounting event as far as proofs pin it; standard TLS/ECDH/AES-GCM/VOLE-ZK hardness; no trusted hardware anywhere.

3. Preliminaries and the provider profile

3.1 Provider profile

The protocol is defined for any upstream satisfying:

1. **TLS 1.3** with an AES-GCM suite in the SHA-256 family (this work: `TLS_AES_128_GCM_SHA256`; `TLS_AES_256_GCM_SHA384` generalizes with SHA-384 gadgets). No compression. No 0-RTT.
2. **HTTP/1.1** with explicit `Connection: close` and `Accept-Encoding: identity` (record-level epoch interleaving requires HTTP/1.1 framing; HTTP/2 would need a different record-mapping).

3. **KeyUpdate accepted mid-request** (client-initiated, `update_not_requested`).
4. **No processing before the complete body.** The provider starts work only once the whole `Content-Length` body has arrived. N relies on this to stop a request whose suffix does not verify (section 4.4).
5. **Parameter pinning semantics**: a JSON request whose *effective* parameters are fully determined by a public suffix that the client can append. The provider must:
   - keep the last value of duplicate top-level keys;
   - let a later object value replace an earlier one rather than merge with it;
   - reject malformed JSON and unknown fields, so the private prefix cannot leave an object or a string open for the suffix to land in.
6. **Cost-relevant parameters overridable by the suffix**: model, token caps (which must cap reasoning tokens too), sampling count, and provider feature flags that change price, including any injected system prompt. The provider must reject any other price-changing field. The size of the prompt (including image or file references) needs no bound here, because the node reserves for a full context window (section 4.4).
7. **A streaming (or terminal) usage event** carrying input/output token counts and, ideally, a provider-computed cost figure, emitted before the response ends.

Items (5) and (6) quantify over every field and input, which no finite test can establish. For the case-study provider (Appendix A) a live regression gate tests all seven items with sampled cases (one unknown field, three unpinned price fields, malformed-JSON variants, both image encodings); the requirements hold as far as those samples show, and the gate is re-run after provider changes. Section 8 and Appendix A.3 discuss other providers. Everything except the specifics of (4)-(7) is provider-generic: the epoch structure, key-schedule proofs, template head, record assembly and proof machinery do not change per provider; only the offer (template, pinned keys, price list, usage parser) does.

3.2 TLS 1.3 mechanics used

Keys. The TLS 1.3 key schedule (RFC 8446 section 7.1) derives, from the handshake secret HS and transcript hashes th_SH (through ServerHello) and th_SF (through server Finished):

$$c_{hs} = \text{HKDF-Expand-Label}(HS, \text{"c hs traffic"}, th_{SH})$$
$$MS = \text{HKDF-Extract}(\text{Derive-Secret}(HS, \text{"derived"}), 0^{256})$$
$$c_{ap_0} = \text{HKDF-Expand-Label}(MS, \text{"c ap traffic"}, th_{SF})$$
$$c_{ap_{i+1}} = \text{HKDF-Expand-Label}(c_{ap_i}, \text{"traffic upd"})$$

with per-epoch AES key K_i and IV IV_i from "key"/"iv" labels, and the GCM tag mask T = AES_{K_i}(J_0), GHASH key H = AES_{K_i}(0^128), and CTR keystream S = AES_{K_i}(J_0+1) || AES_{K_i}(J_0+2) || … where J_0 is the record's nonce-with-counter-1. A record ciphertext is C = P XOR S with tag GHASH_H(AAD || C) XOR T.

KeyUpdate. A sender can rotate its own sending keys at any record boundary; the receiver of the KeyUpdate updates likewise. Sequence numbers restart per epoch.

VOLE-based ZK. We use the interactive QuickSilver-style ZK of TLSNotary's `mpz`, where SHA-256 and AES-128 are provided as gadgets (SHA-256 as a compression-function circuit; AES-128 including key schedule). Both parties are online during the session, so interactivity is free: a proof of the whole key schedule costs tens of SHA-256 compressions and a handful of AES blocks. The VOLE correlation phase is input-independent, so it can be precomputed before a request (section 6.2).

4. Protocol

4.1 Epoch structure

Epoch	Key	Writer	Content
Handshake	handshake secrets	C and U	standard TLS 1.3 handshake
0	c_ap_0	C	one `KeyUpdate(update_not_requested)`
1	c_ap_1	N, then C	N: the credential record (request head with `Authorization`); C: one `KeyUpdate`
2	c_ap_2	C	the JSON request body; its final record is the public suffix, released to U by N only after π_{C2}
Response	s_ap_0	U to C	streamed response; N relays ciphertext

Sequence numbers restart in each epoch. N writes exactly sequence 0 of epoch 1; C therefore starts its epoch-1 sending at sequence 1, and the client-side TLS stack must support exactly this (skip one send sequence; seal-without-send for its epoch-1 KeyUpdate so the relay can order it after the credential record).

4.2 Handshake, relayed and doubly verified

N opens TCP to U and relays bytes. C performs a standard TLS 1.3 handshake with the profile parameters of section 3.1 (one cipher suite, ALPN `http/1.1`, single key share; HRR aborts). After the handshake, C sends N the **handshake** traffic secrets c_hs, s_hs and the transcript hashes. Application secrets cannot be derived from handshake secrets, so this costs nothing to P2.

N decrypts U's encrypted handshake messages, reconstructs the transcript, and verifies (fail-closed): the certificate chain to Mozilla roots and the hostname (P5); CertificateVerify over the transcript; the server Finished; and C's Finished. Both sides thus authenticate U; N additionally knows the transcript hashes used by every later proof.

4.3 Credential record (epoch 1) and the key-material proof

C sends a first KeyUpdate and derives epoch 1. N states the exact plaintext length |head| of its head record (fixed by the published template). C then sends N the **record material** for epoch-1 sequence 0:

- the CTR keystream S truncated to |head|;
- the GHASH key H_1;
- the tag mask T_0 = AES_{K_1}(J_0).

π_{C1} **(C to N, verified before use).** C proves in ZK that this material is correctly derived from the session:

- *private*: the handshake secret HS;
- *public*: th_SH, th_SF, c_hs, s_hs (known to N from the relayed handshake), |head|, and S, H_1, T_0;
- *statement*: the full RFC 8446 chain from HS through c_hs, s_hs, MS, c_ap_0, c_ap_1, K_1, IV_1, and the GCM values above with sequence 0.

N forms the record: P = the published head template with its API key and the client's body length filled in, C_1 = P XOR S, tag = GHASH_{H_1}(AAD || C_1) XOR T_0, and writes header || C1 || tag. N's key bytes never leave N; C never receives C_1.

π_N **(N to C).** N proves that C_1 XOR S equals the published template in which the `{KEY}` field's L positions are constrained to the published alphabet and `{LEN}` equals C's declared body length, and that a public commitment to the record (SHA-256 of `header || C1`) matches; the verifier decodes only the commitment

digest and the alphabet bit. C rebuilds every public input of the statement from the signed offer, its own body length and the S it sent. The proof binds the head's *content* and length. It does not bind the bytes N actually writes: C never sees C_1, and N signs its own attestation, so a different head of the same length is not detectable by C (section 5, residual risk 1).

On success C seals its epoch-1 KeyUpdate (sequence 1) and hands the bytes to N, which writes it immediately after the credential record; C derives epoch 2 and neither side can forge anything in epoch 1 again. N never learns K_1's secret (it has one-shot material for one sequence), and C never receives the key.

4.4 Body (epoch 2)

C writes body = body_private || suffix under epoch 2, with |body| fixed at setup. body_private starts the JSON object and may carry messages, tools, and other schema-valid fields. The suffix is the public string that pins every cost-relevant parameter: model, token caps, n, streaming, usage reporting and provider feature flags. N's TLS relay (the sidecar) builds it from the model and token cap declared at setup and accepts no other. The provider must resolve the effective values to the suffix (section 3.1 (5)).

C sends the suffix as its own final record. N forwards the earlier body records, but **holds the record that completes the body** until π_{C2} verifies. N enforces that the epoch-2 plaintext is exactly |body| bytes, so that record is identifiable. Until it is released, U has an incomplete request and does nothing (section 3.1 (4)). If the proof fails, N closes the connection: no inference ran and nothing was billed. A client therefore cannot run a different model or a larger cap than N reserved for, even briefly.

π_{C2} (C to N, before U can start). C proves that the final bytes of its epoch-2 plaintext equal the agreed suffix:

- *private*: HS;
- *public*: th_SH, th_SF, c_hs, s_hs (anchoring), the record framing (counts and lengths) of the epoch-2 records, the final record's ciphertext, suffix, |body|;
- *statement*: the chain to c_ap_2, K_2, IV_2; CTR-decrypt the final record; check length consistency and that its tail equals suffix. The comparison runs inside the circuit and the verifier decodes a single bit (equal or not), so a failed proof reveals nothing about the decrypted bytes.

Only the final record (the suffix, a few hundred bytes) is decrypted in-circuit, so the cost is about a dozen AES blocks beyond the key schedule, and about 0.4 s on the critical path.

Parameter validation by C. The prover is C, and the public parameters of π_{C1}, π_{C2} and π_{C3} come from N. C checks each against its own view before proving: π_{C1} must cover exactly the material C sent; π_{C2} must be a window ending at the end of C's own final body record (sequence number, offset and ciphertext bytes as C sent them), with C's suffix as the expected value; π_{C3} (section 4.5) must cover the final server records exactly as C received them, matched from the end of the transcript with at most one trailing close_notify record, with C's own disclosure as the expected value. N therefore cannot aim a proof, or its one-bit result, at other plaintext.

Reservation. Because the parameters are fixed before U starts, N reserves funds for the requested model only, at the rates of the offer C verified (section 4.6). The prompt is opaque to N, so the reservation never discounts it: it is $\max(m, \lceil (L \cdot r_{in} + T \cdot r_{out})/1000 \rceil)$ msat, where L is the model's prompt limit (context length minus its completion limit), T the pinned token cap, r_{in}, r_{out} the per-1k rates and m the node's minimum charge. Without a known context length it is the model's undiscounted maximum cost. For the case study (`deepseek-v4-flash`, $L = 104{,}858$, $T = 40$) this reserves 2,282 msat; the session cost 63.

4.5 Response and usage disclosure

U's response streams under s_ap_0; C decrypts and verifies every GCM tag natively (this gives P3 for the response), and streams tokens to the application live.

At close, N signs a **session attestation** binding the session id, certificate chain hash, handshake transcript hash, directional ciphertext hashes, record boundaries and time.

C then discloses R: the plaintext from the boundary of the last usage-bearing data event through the stream terminator, with its chunk framing, anchored at the end of the transcript N relayed.

π_{C3} **(C to N).** C proves that R is the authentic decryption of the final server records:

- *private*: HS; *public*: handshake anchors as above, the final records' ciphertexts and sequence numbers, the disclosure range, a count `skip` of undisclosed bytes after R, and R;
- *statement*: the chain to s_ap_0, K_s, IV_s; CTR-decrypt the covered records; R ends `skip` bytes before the end of the response; in-circuit equality with R, decoded as one bit. For a usage disclosure `skip` is 0.

Because both π_{C2} and π_{C3} anchor on the same public c_hs, s_hs, th_SH, th_SF as the handshake N verified, the disclosed tail is bound to the session. The choice of R is unambiguous: JSON strings cannot contain raw CR/LF, so event boundaries and HTTP chunk framing are never forgeable from model output — a fact we verified by prompting the model to emit fake usage events; every imitation stayed escaped inside a string (Appendix A).

4.6 Settlement

Confidential sessions use the node's ordinary billing; nothing about money is specific to the protocol:

1. **Setup.** C authenticates with the credential it already uses for normal requests, either an API key or a one-off e-cash token that the node turns into one, and names the offer it verified by its signature. N keeps the offers it signed in the last ten minutes and freezes that offer's per-model rates for the session; an unknown or expired offer is refused (HTTP 409) and C refetches it. N reserves on the credential (section 4.4) before any connection to U is opened.
2. **Charge.** N's relay parses R: dechunk, take the usage event and the terminal marker, and require the event's model to equal the setup model; N requires it again before charging. N charges exactly the usage at the frozen rates, $\lceil (r_{in}(p-c) + r_{cached}\min(c,p) + r_{out}\cdot o)/1000 \rceil$ msat for p prompt tokens of which c cached and o completion tokens, capped at the reservation, through the node's ordinary finalize path; a zero cost releases the reservation.
3. **Receipt.** N signs (BIP-340 Schnorr, over the canonical JSON) a receipt with the session id, model, usage, cost, reservation, the resulting balance, the frozen rates, its key, the SHA-256 of the exact attestation bytes, and the time. C requires the signature, takes every field from the signed bytes, and checks the session id and attestation hash against its own session, the usage against the plaintext it already has, and the cost against that usage at the signed rates. Unused balance stays on the credential and is withdrawn through the node's normal refund endpoint.

If C does not disclose after U received the request, N charges the full reservation, which is the reason disclosure is always in C's interest. (Opaque-body flows where the node reads the provider's usage itself can charge nothing instead; here the client controls the disclosure, so charging nothing would make withholding it free.)

Upstream errors. An HTTP-level error from U (for example 503 "overloaded", 429, or a 400 for a request U rejects) carries no usage event. C then discloses only the response *head* as R: the status line, the header lines and the blank line that ends them. π_{C3} proves it with `skip` set to the length of the rest of the response, so R is anchored at a known distance from the end of the transcript and the error body stays private. Error bodies can quote the request: the case-study provider's 400 for an unknown field names the field. N settles from the status alone. For statuses the provider answers before doing any work (4xx, 503), N releases the reservation, so the error costs nothing; other 5xx are charged like a missing disclosure. The head is unambiguous for the same reason as the usage event. A strict status line followed by well-formed `Name: value` header lines (token characters in the name) cannot come from model output, which travels in JSON strings without raw CR/LF, nor from chunk framing, whose lines are bare hexadecimal sizes. The session carries a single response (`Connection: close`). N learns the status line, the response headers and the body length.

4.7 Abort handling

Situation	Outcome
Abort before N releases the final body record (including a failed π_{C2})	U never received a complete request; the reservation is released
Usage disclosed	charged per section 4.6
U returned a 4xx or 503 (disclosed)	reservation released; costs nothing
Released, no usage disclosed (or another 5xx)	full reservation charged (as with a prepaid request); C keeps its transcript and attestation for reputation

5. Security analysis

P1 (credential). The key appears only as plaintext of N's epoch-1 record, written from material that was ZK-proven to be the session's. π_{C1} is verified *before* the material is used, and N assembles the record only from the exact material the proof ran on. Otherwise C could swap in a keystream that rewrites N's head bytes outside the key positions and steer the key into a path or body that the upstream echoes back.

The protection rests on one assumption that must be stated plainly. **C holds the whole epoch-1 key** K_1: it needs it to seal its own KeyUpdate at sequence 1. The key is therefore exactly $C_1 \oplus S$, and P1 holds only because C never sees C_1. Our implementation never sends C_1, the tag or the filled head to C. The commitment N sends is a hash over a 42-character random key, which cannot be brute-forced; the commitment is not salted, so this rests on the key's entropy, and a low-entropy credential would need a salted commitment. The assumption is still about the network, not about cryptography. An observer on the N-to-U path who colludes with C (a shared cloud network, a TLS-inspecting middlebox, a router) learns the key. So does any upstream behaviour that reflects request bytes.

Mitigations, in increasing strength:

1. Keep N's egress path private, and never reflect request bytes in error handling.
2. Use upstream keys with spend caps, rotated often. Where the provider has a key-management API (Venice exposes one for admin keys), mint short-lived, capped keys per node or per hour, so a leaked key is worth one cap.
3. Remove the assumption with a new protocol version in which the TLS handshake runs as a two-party computation, as in TLSNotary, so that neither party ever holds the epoch-1 key. The parties jointly encrypt the credential record and the KeyUpdate and output $c_{ap,2}$ to C alone. This is not a drop-in change to the present protocol; it changes who owns which key and what the proofs may use:
   - C must never learn the ECDHE secret, HS, MS, $c_{ap,0}$ or $c_{ap,1}$, since each yields K_1. π_{C2} and π_{C3}, which take HS as their witness, must be replaced by proofs against public commitments to $c_{ap,2}$ and $s_{ap,0}$.
   - N must not receive the one-shot S, H, T either: that material lets it forge the record. It should receive only the jointly produced ciphertext and the commitments.
   - The specification must fix how the joint transcript, the ciphertext used, the commitments, the KeyUpdate ordering and sequence numbers, and aborts are bound to the session.

   It needs a maliciously secure 2PC functionality and fresh benchmarks covering SHA-256, ECDH and OT, not only an AES-block estimate. MPC would cover the handshake and a few records, not the payload. Until then, the egress assumption is part of P1, and short-lived, capped keys (mitigation 2) limit the loss without giving equivalent security.

P2 (content). After epoch 1, both directions use c_ap_2 / s_ap_0. c_ap_1's one-shot material does not yield c_ap_2 (HKDF), and MS is never derivable from the handshake secrets N holds. C alone holds c_ap_2, hence only C (and U) can read prompt and response. The proofs that decrypt under these keys (π_{C2}, π_{C3}) output one equality bit, on parameters C has checked against its own records (section 4.4), so a malicious N cannot use them as a decryption oracle. What N learns in plaintext is public by design: the suffix, the usage event, or on an error the response head (section 4.6).

P3 (integrity). C's records are authenticated under keys only C has; the head's content is the proven template (its wire bytes are not, residual risk 1); the suffix is proven to be the body's tail, so the effective

(last-wins) parameters are exactly the agreed ones; response GCM tags are verified by C. Tool-call payloads live inside the same authenticated stream.

P4 (accounting). The billed usage is a substring of the authenticated response, proven under the session's server secrets and anchored to the transcript end. N computes the suffix from the setup values, accepts only that exact suffix, and lets U see the complete request only after π_{C2} has verified. U therefore runs exactly the reserved model and cap. The receipt binds the result to the session id and the reservation. Both parties are convinced; outsiders are not (see Non-goals).

P5 (authenticity). Both parties verify the certificate chain, hostname, CertificateVerify and both Finished messages (N from the disclosed handshake secrets; C natively in its TLS stack).

P6 (freshness). Session id = hash(fresh nonces, the node's one-time reservation id); TLS randoms are fresh; the one-shot material is single-epoch, single-sequence.

Residual risks (explicit).

1. π_N *binds content, not wire bytes.* C cannot see the credential record (P1), and N signs its own attestation. N can therefore commit to the honest head and write a different head of the same length, for example another account, another endpoint, or a header that enables provider-side retention. C cannot detect this. The effect is bounded, because C's own records stay authenticated and a broken head shows up as a failed response, but it is not "disputable" after the fact. A 2PC protocol version (P1, mitigation 3) would close it.
2. *Traffic analysis.* N sees the size of every record. The body length is visible, and so is the response's *token-length sequence*, because streaming providers emit one record per token or small token group. This side channel has been shown to reconstruct a large share of LLM responses (Weiss et al., USENIX Security 2024). Two comparisons matter:
   - Against the existing Routstr design, where the node sees the full plaintext, this is strictly less exposure: lengths instead of content.
   - Against "N sees ciphertext only", it is more than "volume and timing". C cannot pad U's records. The prompt side can be padded inside the JSON body. Real mitigation needs provider-side padding or batching of streamed chunks, which some providers already apply.
3. *DoS and limits.* The node and its relay validate the setup and reserve before opening any connection to U; they bound concurrent sessions (32 each), the time to setup (15 s), the session lifetime (600 s) and the body (8 MiB). Per-client rate limits remain a deployment matter.
4. *Error metadata.* On an upstream error N learns the status line, the response headers and the body length, though not the body.
5. *Client and node hygiene.* Keys live in C's process; the node holds the API key in memory. The protocol does not protect against a compromised endpoint. C's certificate validation is RFC 5280 path validation to the WebPKI roots, without revocation or certificate-policy checks.
6. *Interactivity.* Proofs are interactive and there is no artifact a third party can check. The node is online during the session, so this is free. A non-interactive proof backend would not by itself give one: C holds $s_{ap,0}$ and can forge server records, so a portable artifact needs an independent transcript attestation, or MPC-TLS, where no single party holds the server keys.
7. *Provider drift.* The profile (section 3.1) must be regression-tested per provider; a provider change that breaks parameter pinning or usage reporting degrades a *verified* session into an unverifiable one, which clients must treat as failure, not fallback.
8. *Balances still rest on N's goodwill after an abort.* If N cuts the stream before the usage event, it charges the reservation, and C cannot prove who aborted. Returning unused balance is also N's ordinary refund. This is the same trust as in the existing Routstr design, where the node reports usage and returns unused balance on goodwill and reputation. The difference is in the honest case:
   - cost is computed from upstream-authenticated usage, at the signed rates the client checks;

- unbilled upstream errors and sessions that never reached U cost nothing, verifiably;
- the receipt binds all of it.

The amount at N's discretion is the reservation for the requested model only.

6. Implementation and evaluation

6.1 Prototype

Implemented: `cu-crypto` (native key schedule, GHASH, RFC 8448/NIST-vector-tested), `zkbench` (mpz2 proofs for π_{C1}, π_N, π_{C2}, π_{C3}; native==ZK, mutation and one-bit-disclosure tests), `cu-sidecar` (relay, fail-closed handshake verification, record assembly, attestation), and a reference TypeScript client spawning the Rust prover, which receives its job and witness on stdin, so the handshake secret never touches the disk. The client's TLS stack is a fork of Reclaim's TLS 1.3 client adding handshake-secret export, send-sequence skipping and seal-without-send. It accepts only the version and suite it offered and validates certificate paths per RFC 5280 (CA and path-length constraints, key usage, extended key usage, name constraints, unknown critical extensions), matches hostnames per RFC 6125, uses the full 64-bit record sequence in the nonce, compares Finished values in constant time, rejects duplicate extensions and a server KeyUpdate that requests a reply, and does not fetch intermediates from AIA URLs.

6.2 Costs and optimizations

Proof costs (Apple M2-class host; `local` = in-process transport, `wan` = 25 ms one-way delay, 100 Mbps token bucket):

Statement	Wall (local / wan)	Prover to V	V to Prover	SHA-256 compressions	AES blocks
π_{C1} (record material)	0.47 s / 1.53 s	574 kB	1.09 MB	32	18
π_N (head template)	0.09 s / 0.76 s	464 kB	484 kB	4	0
π_{C2} (suffix)	0.47 s / 1.53 s	576 kB	1.09 MB	36	6
π_{C3} (usage disclosure)	0.47 s / 1.54 s	579 kB	1.09 MB	28	38

End-to-end, versus a direct HTTPS call to the same provider with the same body (medians, client, node and sidecar on one host, real upstream). Upstream latency itself drifted between about 0.8 and 1.5 s during the runs, so every comparison below interleaves the two paths.

Run (60-token streamed answer)	Direct TTFT	Protocol TTFT	Added
Client daemon + OpenAI client, cold, interleaved, n=8	731 ms	1798 ms	**+1067 ms**
Reference client, pre-warmed, n=5 (timed from claim)	731 ms	985 ms	**about +0.25 s**

The cold path breaks down as about 0.68 s before the body (relayed handshake, π_{C1}, π_N, credential record) and about 0.39 s holding the final body record while π_{C2} verifies. Pre-warming moves the first part into the arming step; the π_{C2} hold depends on the prompt and stays.

Streaming the remaining tokens through the relay is also slower than direct: 1.0 to 1.4 s instead of about 0.75 s for 60 tokens in these runs. π_{C3} and settlement do not add to this: the client's stream ends as soon as the authenticated response is complete, and they finish in the background.

These are best cases. With client and node on different continents, every relayed handshake round trip and each interactive proof adds network delay (π_{C1} alone takes 1.53 s at 25 ms one-way delay, table above).

π_{C1} and π_{C2} are on the critical path; π_{C3} runs after the response.

Optimization path (integration branches). (a) The VOLE/OT correlation of each proof is input-independent and can be computed before the request arrives; precomputing it removes the setup share of the 0.47 s. (b) Client daemons that hold a balance/account with a node can keep *pre-warmed sessions*: the TLS handshake with the upstream and the OT correlation are done ahead, so an arriving request only runs the request-bound part of π_{C1}, the credential record and the body — with a pool refilled after each request. (c) The mpz2 prover's rayon feature, opt-in (`--features rayon`), parallelizes compression circuits. Measured: with (b) the request path is only the KeyUpdate, the credential record and the body (claim to body write about 2 ms), and π_{C1}/π_N move into the arming step. (a) turned out to be unnecessary once π_{C1} is armed, and the mpz2 version used exposes no separate preprocessing. An armed session holds a reservation on the client's credential until it is claimed or discarded (released). The reference client implements arming; the client daemon does not pre-warm yet.

6.3 Live verification

Live sessions against the case-study provider (2026-10-10) completed with all four proofs verified, a signed receipt passing every client check, and the cost within the reservation (63 of 2,282 msat in the example of section 4.4). They were billed through the node's ordinary accounts, with both an API key and a one-off e-cash token as the credential. An upstream 400 was settled at zero cost with only the response head disclosed: the provider's error body quoted the request's private field name, and neither the node's nor the relay's logs contained it. A client daemon ran confidential requests with an off-the-shelf OpenAI client, streaming and non-streaming, through its normal provider selection, credential and balance accounting; the node logged no plain chat request, and unsupported endpoints were refused with HTTP 400. Five idle connections that never sent a setup caused no connection to the provider and were closed at the setup timeout.

Adversarial runs are a regression gate (`crypto/tools/adversarial-model-swap.sh`) with an honest control and an expected outcome per case, checked against the node's billing log; it exits non-zero on any deviation. In the live run the control settled, a decoy model in the private part settled as the setup model, and every attack was refused with the reservation released:

- a suffix other than the canonical one;
- a body naming a pricier model, or a model the node does not offer, with the suffix withheld: no response byte, U never received a complete request;
- a body ending in another model's suffix while claiming the canonical one: π_{C2} fails.

The relay also refuses, as verified in live runs, a client that discloses the handshake secret, a second disclosure or second material during a running proof, and extra client records before the credential record or past the declared body length.

7. Deployment pattern (negotiation, billing, compatibility)

Confidential mode plugs into an existing routing deployment as an alternative *transport*. Everything else stays the deployment's own code. Negotiation is per request and stateless:

1. **Offer.** The node publishes a signed offer (version range, upstream hostname, key length and alphabet, template, pinned suffix keys, a signed rate snapshot, attestation key) at a public endpoint.
2. **Model listing.** The models listing gains one additive field per model referencing the offer; old clients ignore it.

3. **Opt-in.** A client opts in per request, for `POST /v1/chat/completions` only; any other endpoint is refused before a credential or deposit is used. It then routes only to nodes that advertise the field for the model (only models with a signed price entry are advertised), and requires that:
   - the offer signature verifies with a pinned node key, not just the key the offer itself names (an offer that only verifies against its own embedded key proves self-consistency, not authenticity);
   - the upstream hostname matches the client's **trusted hostname list (wildcards supported)**, so a malicious node cannot route a confidential session to an unexpected host; an empty or missing list refuses every host;
   - the prover is available.

   The client also rebuilds every public input of π_N from the signed offer rather than taking it from N, and re-applies these checks under each request's own policy when it reuses a cached offer.
4. **Billing** is the node's ordinary billing (section 4.6). The credential is the one the client already holds for the node; the reservation, settlement, balance and refunds are the existing ones. A client daemon deposits enough for the full-context reservation (the model's maximum prompt cost plus the completion cap, with a 5% margin, capped at the model's maximum cost). In the reference integration:
   - the node adds a websocket route and an offer;
   - the client SDK adds one transport branch beside its other request transports, so credential acquisition, top-ups, failover, usage tracking and balance accounting are unchanged;
   - the daemon adds a configuration block and passes the opt-in through.
5. **Response.** The client streams the decrypted body to the application as records arrive. If the caller did not ask for a stream, it folds the SSE into a single completion object, since the pinned suffix always requests streaming because the usage event only exists there. Checks that complete after the response (the π_{C3} disclosure, the receipt's signature and binding, the usage, the cost, the served model) are not dropped: a failure fails a non-streaming call and ends a stream with an error instead of a clean end, and an abort cancels the session.
6. **Model selection.** Nodes may restrict confidential mode to text-only models. The ordinary reservation already covers a full context window, so this is a conservative choice, not a billing requirement.

For existing deployments the change is purely additive: new endpoints and fields only. The ordinary proxy path, direct API-key clients and web interfaces are untouched, and old client daemons never see the new fields. Requests that opted in fail closed: the daemon answers HTTP 400 when confidential mode is not enabled or the endpoint is not chat completions, and never sends such a request on the plain path, since a silent downgrade would break the privacy promise. The node and its relay refuse every invalid or over-limit session with an explicit error frame carrying an HTTP-like status, and settle each session exactly once; if the settlement write fails, the node charges the reservation when the request reached U and releases it otherwise.

8. Comparison with alternative approaches (at the end)

TLSNotary-style MPC-TLS ("red's approach", Design A in the spec). C and N run an interactive 2PC during the TLS handshake and record processing; C proves what was sent/received, and the API key can be N's private input with selective disclosure of the usage event. Properties: no trusted hardware; content hidden from N *if* roles are flipped (C prover, N verifier) as in Design A; strong integrity; no live streaming with deferred decryption; all payload processing happens inside MPC. Our measurements of the MPC path: 2 KB request about 5.8 s and 41.5 MB transferred; 32 KB about 34 s / 377 MB; 128 KB about 129 s / 1.44 GB (100 Mbps, 50 ms emulated). The cost scales with payload size, and the protocol has no streaming mode, so it suits short prompts only. Relative to this paper's design: MPC-TLS needs no KeyUpdate semantics from the provider (works with TLS 1.2) and gives third-party-verifiable transcripts, at roughly 5× the latency and 7× the transfer of ours for a 2 KB request (5.8 s and 41.5 MB against about 1.1 s and 5.9 MB) and about two orders of magnitude at 32-128 KB; ours needs a TLS 1.3 provider profile but streams with about 1 s of proof and relay overhead before the first token.

Proxy zkTLS attestors (the "verified mode" deployed on nodes; Reclaim-style attestors). The node is the TLS client; a notary/verifier checks the node's transcript for request equality. Properties: no hardware; the notary (and the node) see plaintext — the prompt is *not* hidden from the node; proofs are portable; costs

are seconds and hundreds of MB for large transcripts, and per-request latency overhead measured at +10.8 s TTFT on a live node. It solves request/response integrity and not confidentiality.

TEE (enclave) designs. The node runs in a hardware enclave that holds the API key and processes the prompt in the clear but is attested; users verify attestations. Properties: fast (native speed), streaming, prompt hidden from the operator; trust in the hardware vendor and the enclave code measurement; attestation lifecycle must be maintained as code updates ("re-attest after every update" for clients); no cryptographic receipt for third parties beyond the operator's signature.

	This work	MPC-TLS (TLSNotary)	Proxy zkTLS	TEE
Prompt hidden from node	yes	yes (flipped roles)	no	yes
API key hidden from client	yes (under the P1 egress assumption, §5)	yes	yes	yes
Trusted hardware	no	no	no	yes
Streaming, native-speed payload	yes	no	yes	yes
Added latency (this workload)	~1.1 s cold, ~0.3 s pre-warmed (local); more over WAN	5.8-129 s	~10 s	~0
Transfer overhead	~5.9 MB/session (both directions) plus relayed TLS	41 MB-1.4 GB	transcript size x2	~0
Third-party-verifiable transcript	no (non-goal; two-party proofs)	yes	yes	attestation only
Provider requirement	TLS 1.3 profile (3.1)	TLS 1.2+	TLS 1.2/1.3	any
Model binding	suffix + proofs	selective disclosure	comparator heuristics	enclave code

The approaches compose: proxy zkTLS today provides integrity for unmodified providers; MPC-TLS is the fallback for providers that fail the profile; a TEE remains an alternative trust model, not a replacement for proofs. Our design is best understood as the fast lane where a provider's TLS and JSON semantics cooperate.

9. Conclusion

We presented a protocol that gives LLM routing users prompt confidentiality, request/response integrity (with the request head bound to its template content, not its wire bytes) and upstream-anchored accounting against a node that only relays ciphertext, with no trusted hardware and no MPC over the payload. The node's API key stays hidden from the client as long as no observer of the node's egress traffic colludes with the client (section 5). The mechanisms are a one-shot, ZK-proven TLS record for the node's credential, two key rotations to strand it, a proven public suffix for parameter pinning, and a proven usage disclosure for settlement. A complete prototype, verified against a live provider with all proofs checking, adds about a second to the first token cold and about 0.3 s with pre-warmed sessions. The design is provider-parameterized: its provider profile turns "is this transferable to provider X?" into a checklist, which we apply in Appendix A.

Appendix A. Case study: the provider profile in practice

A.1 Verified behaviours (live gate `crypto/tools/provider-profile-check.py`, checks A1-A11, 11/11 on 2026-10-10; rate limits, 5xx and timeouts count as inconclusive, never as a pass, and each rejection check asserts the exact status and error text against a valid control).

- **Transport:** a client offering only TLS 1.3 with `TLS_AES_128_GCM_SHA256` is served; a client KeyUpdate sent in the middle of the request body is accepted and the server answers completely and closes (`Connection: close`, HTTP/1.1).
- **Waits for the full body:** nothing for 8 s with the body incomplete, then a response within 0.7 s of the last bytes.
- **Last-wins:** a bad model then a good one gives 200; a good one then a bad one gives 404; a `max_tokens` of 200 followed by 8 behaves like 8.
- **Replace, not merge:** a later `venice_parameters` object replaces an earlier one; an invalid `character_slug` that is overridden is ignored.
- **Strict parsing (sampled):** an unknown top-level field and malformed JSON are rejected with 400, including a suffix landing in an open object or string. The 400 body for an unknown field quotes the field's name, which is why an error disclosure covers only the response head (section 4.6).
- **Token cap:** `max_completion_tokens` caps output including reasoning, with a small overshoot on tiny caps (3 billed up to 6, 40 billed 39). The reservation covers this easily, because its prompt side is sized for a full context window.
- **Input bound:** with the pinned `include_venice_system_prompt: false` a 224-byte request billed 6 prompt tokens. Without the pin Venice injects about 1.6k tokens of system prompt.
- **Unpinned price fields (sampled):** `service_tier`, `web_search_options` and `prediction` are rejected.
- **Images:** every offered model rejects image parts, both `image_url` and data URIs (`crypto/tools/regression-modalit` 8/8).
- **Usage event:** the terminal event carries usage, cost and model with empty `choices`, followed by `[DONE]`.
- **Fake usage events:** a prompt instructing the model to emit them produced none. Every imitation was escaped inside a JSON string, and the raw stream contained no CR/LF inside events.

A.2 Profile application. The pinned suffix for this provider: `,"model":M,"max_tokens":T,"max_completion_tokens":T,` with `max_completion_tokens` pinned because it overrides `max_tokens` (a bypass observed in testing) and the provider-specific parameters pinned to keep web-search and system-prompt injections off the bill. The regression gates of A.1 re-check the profile after upstream changes.

A.3 Other providers. The generic pieces (epochs, π_{C1} to π_{C3}) port unchanged. The open questions per provider are exactly profile items (4)-(6): duplicate-key/unknown-field semantics, streaming usage availability, and overridability of cost-relevant parameters; plus whether HTTP/1.1 and mid-request KeyUpdate are still accepted (some CDNs pin HTTP/2 and may reject KeyUpdate). Where a provider fails, the fallbacks are the MPC-TLS variant (section 8, second column) for confidentiality, or the existing integrity-only path. Appendix A.3 is expanded with per-provider assessments in the project's `docs/ADVERSARIAL-QA.md`.